Entrega 1 — Alcance y descomposición
Proyecto Integrador · Módulo: **Clientes y Trabajos**

Sistema de gestión para un plomero-gasista. Este documento cubre solo el módulo de clientes y trabajos. Presupuestos y turnos están a cargo de otros grupos.

1. Alcance del módulo

Qué hace

- Registrar un cliente nuevo (nombre, teléfono, dirección, email opcional).
- Modificar los datos de un cliente existente.
- Buscar un cliente por nombre o teléfono y listar todos los clientes.
- Registrar un trabajo asociado a un cliente (tipo, descripción, dirección, fecha de alta).
- Cambiar el estado de un trabajo: **pendiente** → **en curso** → **finalizado**, o **cancelado**.
- Consultar los trabajos de un cliente y listar trabajos por estado.
- Permitir que otros módulos consulten un cliente o un trabajo por su identificador.

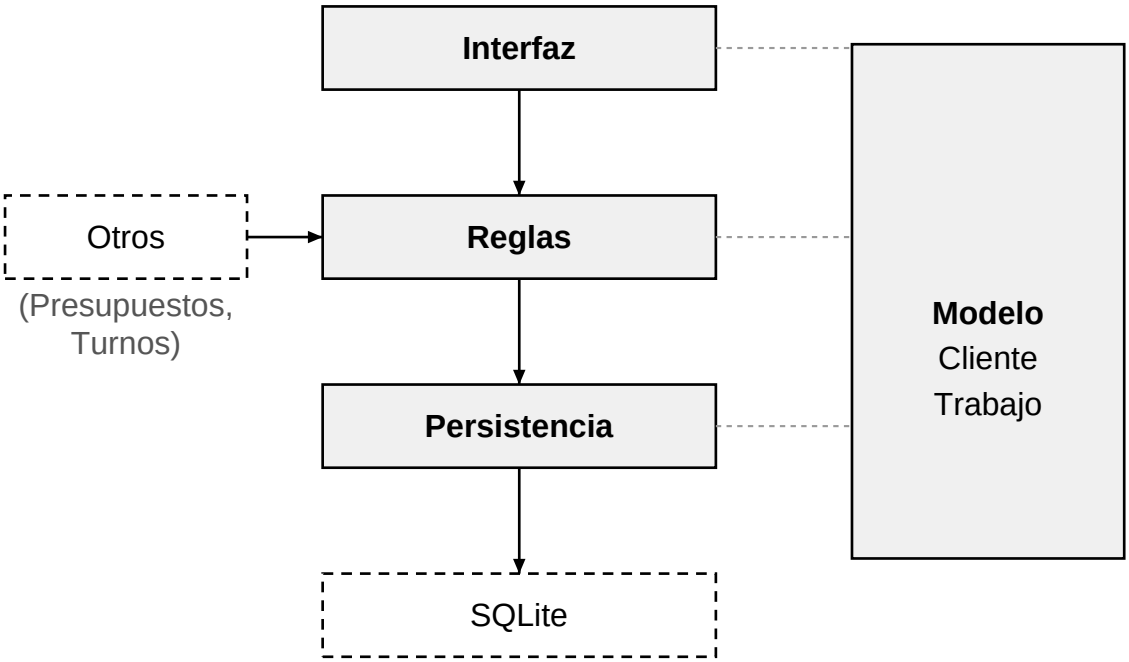
Qué no hace

Queda afuera	Motivo
Presupuestos y cálculo de costos	Es otro módulo del proyecto.
Turnos y agenda	Es otro módulo del proyecto.
Eliminar clientes o trabajos	Se conserva el historial. Un trabajo se **cancela**, no se borra.
Facturación y pagos	Excede el tiempo disponible.
Usuarios, login y permisos	Se asume un único usuario (el plomero).
Fotos y archivos adjuntos	Excede el tiempo disponible.

2. División en partes

Parte	Responsabilidad	Necesita de
Modelo	Representar qué es un cliente y qué es un trabajo, con sus estados posibles.	Nada.
Reglas	Validar los datos y controlar qué cambios de estado de un trabajo son válidos.	Modelo, Persistencia.
Persistencia	Guardar y recuperar clientes y trabajos en SQLite.	Modelo.
Interfaz	Pedir datos al usuario y mostrar clientes y trabajos por pantalla.	Reglas, Modelo.

3. Esquema



Las flechas llenas indican **“usa a”**. Las líneas punteadas hacia Modelo indican que las tres partes intercambian objetos Cliente y Trabajo, sin depender de cómo se guardan o se muestran.

4. Justificación

¿Por qué dividimos así?

Cada parte cambia por un motivo distinto: la pantalla (Interfaz), las reglas del negocio (Reglas), el almacenamiento (Persistencia) y la forma de los datos (Modelo). Eso da **alta cohesión**, porque cada módulo se puede describir con una frase corta. Mantuvimos clientes y trabajos juntos dentro de cada parte, y no en dos módulos separados, porque un trabajo siempre depende de un cliente y separarlos duplicaría capas y aumentaría el acoplamiento entre ellos. Reglas no sabe cómo se guarda ni cómo se muestra nada: recibe datos y devuelve resultados, lo que da **bajo acoplamiento** y permite probarla sin base de datos ni pantalla.

Si cambiáramos SQLite por otra base de datos

Habría que tocar **solo Persistencia**. Es la única parte que conoce SQL y la estructura de las tablas. Reglas la usa a través de funciones como guardar o buscar un cliente, que reciben y devuelven objetos del Modelo, y la Interfaz no sabe que existe una base de datos. Mientras la nueva Persistencia mantenga las mismas funciones, el resto no se modifica.

¿Qué parte será la más difícil?

Creemos que **Persistencia**. Tiene que representar la relación entre cliente y trabajo (cada trabajo referencia a un cliente), convertir filas de la base en objetos del Modelo y viceversa, y mantener todo eso sin filtrar detalles de SQL hacia Reglas. Cualquier error acá se propaga al resto del módulo.